

Essa é uma das partes MAIS importantes do JavaScript.

Sem isso você não consegue entender direito:

- APIs
- fetch
- banco de dados
- React
- Node.js
- timers
- sistemas modernos
- requisições HTTP

O que significa assíncrono?

Assíncrono significa:

| algo que não precisa esperar terminar para o resto do código continuar.

Exemplo da vida real

Imagine:

Você pediu uma pizza.

Síncrono

Você fica parado esperando a pizza chegar.

Não faz mais nada.

Assíncrono

Você pede a pizza e continua:

- estudando
- jogando
- trabalhando
- tomando banho

Quando a pizza chega:

```
"Ei, terminou!"
```

JavaScript é Single Thread

JavaScript executa:

```
uma coisa por vez
```

Ele possui apenas uma thread principal.

Então como JavaScript faz várias coisas?

Usando:

- Web APIs
 - Event Loop
 - Callback Queue
-

Primeiro exemplo assíncrono

```
console.log("Início")

setTimeout(() => {

  console.log("Executou depois")

}, 2000)
```

```
console.log("Fim")
```

Resultado

```
Início  
Fim  
Executou depois
```

Por que isso aconteceu?

Porque:

```
setTimeout()
```

é assíncrono.

Ele NÃO bloqueia o código.

O que acontece internamente?

PASSO 1

JavaScript executa:

```
console.log("Início")
```

PASSO 2

Encontra:

```
setTimeout()
```

PASSO 3

Entrega o timer para o navegador (Web API).

O navegador começa a contar:

```
2000 ms
```

PASSO 4

Enquanto isso o JavaScript continua.

Executa:

```
console.log("Fim")
```

PASSO 5

Depois de 2 segundos:

A função fica pronta.

PASSO 6

Ela vai para:

```
Callback Queue
```

PASSO 7

O Event Loop espera a pilha principal ficar vazia.

Então executa:

```
console.log("Executou depois")
```

Visualização interna

CALL STACK

console.log("Início")

setTimeout()

console.log("Fim")

↓

WEB APIs

timer de 2 segundos

↓

CALLBACK QUEUE

função pronta

↓

EVENT LOOP

espera a stack ficar vazia

Callback

Antes de Promise existir, usávamos callbacks.

O que é callback?

É:

| uma função passada para outra função.

Exemplo simples

```
function saudacao(nome, callback) {  
  
    console.log("Olá " + nome)  
  
    callback()  
  
}  
  
function terminou() {  
  
    console.log("Terminou")  
  
}  
  
saudacao("Gabriel", terminou)
```

Resultado

```
Olá Gabriel  
Terminou
```

Callback assíncrono

```
setTimeout(() => {  
  
    console.log("Passaram 2 segundos")  
  
}, 2000)
```

Problema dos callbacks

O código começava a ficar assim:

```
login(usuario, () => {  
  
  buscarPerfil(() => {  
  
    buscarPosts(() => {  
  
      buscarComentarios(() => {  
  
        console.log("fim")  
  
      })  
  
    })  
  
  })  
  
})  
  
})
```

Isso ficou conhecido como:

Callback Hell

Porque o código ficava difícil de ler.

Promise

Promise significa:

```
promessa
```

Uma Promise promete:

```
"vou te entregar um resultado no futuro"
```

Estados da Promise

Estado	Significado
pending	esperando
fulfilled	sucesso
rejected	erro

Criando uma Promise

```
const promessa = new Promise((resolve, reject) => {  
  
  let sucesso = true  
  
  if (sucesso) {  
  
    resolve("Funcionou")  
  
  } else {  
  
    reject("Erro")  
  
  }  
  
})
```

O que é resolve?

Significa:

deu certo

O que é reject?

Significa:

deu errado

Consumindo Promise

```
promessa
  .then((resultado) => {

    console.log(resultado)

  })
  .catch((erro) => {

    console.log(erro)

  })
```

Resultado

Funcionou

Promise com tempo

```
const promessa = new Promise((resolve) => {

  setTimeout(() => {

    resolve("Dados carregados")

  }, 2000)

})

promessa.then((dados) => {

  console.log(dados)
```

```
} )
```

Resultado depois de 2 segundos

Dados carregados

then()

Executa quando a Promise dá certo.

catch()

Executa quando a Promise dá erro.

Async / Await

Depois surgiu uma forma mais limpa.

async

Transforma a função em assíncrona.

await

Espera uma Promise terminar.

Criando função assíncrona

```
function carregarDados() {  
  
  return new Promise((resolve) => {  
  
    setTimeout(() => {  
  
      resolve("Dados recebidos")  
  
    }, 2000)  
  
  })  
  
}
```

Usando async/await

```
async function executar() {  
  
  console.log("Carregando...")  
  
  const resultado = await carregarDados()  
  
  console.log(resultado)  
  
}  
  
executar()
```

Resultado

```
Carregando...  
Dados recebidos
```

O que o await faz?

Ele fala:

"espere essa Promise terminar"

IMPORTANTE

`await`

só funciona dentro de:

`async function`

Exemplo real com API

```
async function buscarUsuario() {  
  
  const resposta = await fetch(  
    "https://jsonplaceholder.typicode.com/users/1"  
  )  
  
  const dados = await resposta.json()  
  
  console.log(dados)  
}  
  
buscarUsuario()
```

O que acontece aqui?

fetch()

Faz uma requisição para a internet.

await fetch()

Espera a resposta chegar.

resposta.json()

Transforma JSON em objeto JavaScript.

Tudo isso é assíncrono.

Tratando erros com try/catch

```
async function carregar() {  
  
  try {  
  
    const resposta = await fetch("url-invalida")  
  
    console.log(resposta)  
  
  } catch (erro) {  
  
    console.log("Erro encontrado")  
  
  }  
  
}
```

Fluxo moderno mais usado

```
Usuário faz ação  
    ↓  
fetch()  
    ↓  
Promise  
    ↓  
await
```

↓
Dados chegam
↓
Tela atualiza

Exemplo completo REAL

```
async function buscarPosts() {  
  
  try {  
  
    console.log("Buscando posts...")  
  
    const resposta = await fetch(  
      "https://jsonplaceholder.typicode.com/posts"  
    )  
  
    const posts = await resposta.json()  
  
    console.log(posts)  
  
  } catch (erro) {  
  
    console.log("Erro ao buscar posts")  
  
  }  
  
}
```

buscarPosts()

O que esse código faz?

1. Faz requisição na internet
2. Espera resposta chegar
3. Converte JSON em objeto
4. Mostra os dados
5. Se der erro → catch executa

Resumo Geral

Conceito	Explicação
síncrono	espera terminar
assíncrono	continua executando
callback	função chamada depois
Promise	promessa de resultado futuro
resolve	sucesso
reject	erro
then	executa quando der certo
catch	executa quando der erro
async	cria função assíncrona
await	espera Promise

Evolução do JavaScript Assíncrono

```
Callbacks
  ↓
Promises
  ↓
Async / Await
```

O mais usado hoje

✅ Async / Await

Porque o código fica:

- mais limpo
- mais organizado
- mais parecido com código normal
- mais fácil de manter

Frase FINAL para lembrar

Assíncrono = JavaScript não precisa parar tudo enquanto espera algo terminar.

Mini exercício

Tente criar:

1. Uma Promise usando setTimeout
2. Esperar ela com await
3. Mostrar uma mensagem depois de 3 segundos

Exemplo esperado:

```
Esperando...  
Terminou!
```